



1. What is MVC?

MVC euta architectural pattern ho, jasle application ko logic lai 3 main component ma bhibhajit garchha:

- **M - Model:** Data Layer. Database ko structure, tables, collections, ra fields kasari baschhan bhanera model le rule set garchha. [\[1, 2, 3, 4\]](#)
- **V - View:** UI Layer (Optional in Backend). User le mobile ba browser ma dekhne graphical interfaces हो। Jaba hami REST API banauchhaũ, view ko thau ma hami **JSON Data Response** pathauchhaũ. [\[1, 2, 3\]](#)
- **C - Controller:** Brain/Logic Layer. Client bata aako request bujhne, data manipulate garne, ra final response pathaune main processor Controller हो। [\[1\]](#)

[Client Request] —> [Routes] —> [Controllers] <—> [Models (DB)]
|
|> [Sends Response (JSON)]



2. Clean Project Structure (The Industry Standard)

Euta professional backend app ko folder structure yesri split garienchha:

text

```
my-mvc-app/
├── config/                # Database ra third-party service connections
│   └── db.js
├── models/               # Database schemas
│   └── userModel.js
├── controllers/          # Core business logic handlers
│   └── userController.js
├── routes/               # Express endpoints routing
│   └── userRoutes.js
├── services/             # Reusable business logic (heavy operations)
│   └── userService.js
├── utils/                # Small helper/utility functions
│   └── formatHelper.js
├── .env                  # Environment variables
├── index.js              # Application entry point
└── package.json
```

Use code with caution.

3. MVC Components in Action (Step-by-Step Code)

Aba hami euta user management module banayera real-world file management herchhaũ:

Config Folder (config/db.js)

Database setup code halne thau:

javascript

```
// Database server connection details
const connectDatabase = () => {
  console.log("Database successfully connected dynamically!");
};
module.exports = connectDatabase;
Use code with caution.
```

Models (models/userModel.js) [1]

Database bhitra data ko skeleton kasari baschha schema declare garne:

javascript

```
// Yo real data layer ko representation ho (Dummy DB representation)
const usersTable = [
  { id: 1, name: "Sita", role: "admin" },
  { id: 2, name: "Gopal", role: "user" }
];

module.exports = usersTable;
Use code with caution.
```

Services (services/userService.js)

Heavy database queries, external API call, ba reusable heavy calculations controller ma narakhi unique services folder ma handle garienchha. [1, 2]

javascript

```
const usersTable = require('../models/userModel');

// Database logic operation separate thau ma
const fetchUserFromDB = (id) => {
  return usersTable.find(u => u.id === parseInt(id));
};

module.exports = { fetchUserFromDB };
Use code with caution.
```

Controllers (controllers/userController.js)

Request line, response dine ra flow validation garne kaam:

javascript

```
const userService = require('../services/userService');

const getUserDetails = (req, res) => {
  try {
    const userId = req.params.id;
    const user = userService.fetchUserFromDB(userId); // Service call
    gareko

    if (!user) {
      return res.status(404).json({ success: false, message: "User
bhetiyenan" });
    }

    res.status(200).json({ success: true, data: user });
  } catch (error) {
    res.status(500).json({ success: false, message: "Server crash
error" });
  }
};

module.exports = { getUserDetails };
Use code with caution.
```

Routes (routes/userRoutes.js) [\[1\]](#)

Routes file le business logic handling ko barema kura jandaina, yesko single responsibility controller endpoints pointer map garne matra ho.

javascript

```
const express = require('express');
const router = express.Router();
const userController = require('../controllers/userController');

// URL map strictly to Controller logic
router.get('/:id', userController.getUserDetails);

module.exports = router;
Use code with caution.
```

Utilities (utils/formatHelper.js) [\[1\]](#)

Sajilo helpers haru like Date format, text uppercase capitalization logic, password encryption formatters jasta dynamic short utils function handle garna:

javascript

```
const capitalizeText = (text) => {
  return text.toUpperCase();
};

module.exports = { capitalizeText };
Use code with caution.
```

4. Linking Everything in Entry Point (`index.js`)

Aba mathika sabai components lai main initialization logic standard application layer ma integrate garū:

javascript

```
const express = require('express');
const connectDatabase = require('./config/db');
const userRoutes = require('./routes/userRoutes');

const app = express();
const PORT = 3000;

app.use(express.json());

// Initialize Core Database Configurations
connectDatabase();

// Register Clean Component Routes
app.use('/api/users', userRoutes);

app.listen(PORT, () => {
  console.log(`Enterprise MVC Server running on port ${PORT}`);
});
```

Use code with caution.

Essential Phase Review:

1. **Separation of Concerns:** Component separate bhaye pachi, model bigrida controller code badli rahana pardaena.
2. **Readability:** Thulo teams haru ma kaam garda direct single module target garera hotfixes pathauna safe hunchha.
3. **Slim Controllers:** Controller lai safe ra short banauna reusable data structures haru **Services** framework ma handle garna parcha.